

TaskFlow — Aplicación de Gestión de Tareas

Proyecto de la evaluación del módulo "Programación avanzada en JavaScript" (Alkemy).

Cómo ejecutarlo

Abrí index.html en el navegador (no requiere instalación ni servidor).

Estructura

- index.html — formulario para crear tareas, filtros y contenedor de la lista.
- style.css — estilos de la interfaz.
- script.js — toda la lógica de la aplicación.

Informe breve

1. Orientación a objetos

- Tarea: clase con id, descripcion, estado, fechaCreacion y fechaLimite. Incluye cambiarEstado() (alterna pendiente/completada), eliminar() (la propia tarea se marca como eliminada) y tiempoRestante().
- GestorTareas: administra el arreglo de tareas (agregarTarea, eliminarTarea, cambiarEstadoTarea, obtenerTareas con filtro, cargarTareas). eliminarTarea delega en tarea.eliminar() y luego filtra las marcadas como eliminadas.

2. ES6+

Se usa let/const en todo el código, template literals para armar textos y fechas, arrow functions en callbacks y helpers, y destructuring/spread/rest (por ejemplo al leer el formulario, al clonar el arreglo de tareas y al construir mensajes de notificación con ...detalles).

3. Eventos y DOM

- submit del formulario para agregar tareas.
- click delegado en la lista para completar (tarea__check) o eliminar (tarea__eliminar).
- mouseover/mouseout para resaltar la tarea bajo el cursor.
- keyup en el input para agregar con Enter.
- Filtros (Todas / Pendientes / Completadas) que re-renderizan la lista.

4. Asincronía

- agregarTareaConRetardo simula latencia con setTimeout (600ms) antes de insertar la tarea.
- Una notificación aparece 2 segundos después de crear la tarea (setTimeout anidado).
- iniciarContadorGlobal usa un setInterval que corre cada 1 segundo y actualiza, para cada tarea con fecha límite, un contador regresivo real con formato "Vence en: Xd Xh Xm Xs" (formatearContador + actualizarContadorDeTarea). El contador se pone amarillo cuando falta menos de 1 hora y rojo con "¡Vencida!" al pasar el límite.

5. Consumo de APIs

- importarTareasDesdeAPI hace fetch a JSONPlaceholder (GET /todos) para traer tareas de ejemplo, con try/catch para errores de red o HTTP.
- sincronizarConAPI simula un POST de las tareas actuales a la API.
- guardarEnLocalStorage / cargarDeLocalStorage persisten y recuperan las tareas del navegador, reconstruyendo instancias de Tarea a partir del JSON guardado.

Posibles mejoras futuras

- Edición inline de la descripción de una tarea.
- Ordenar tareas por fecha límite.
- Sincronización real con un backend propio en lugar de la API de ejemplo.